



TECHNICAL BRIEF • 2026

Architecture, security & standards alignment.

A reference document for a software engineer evaluating 50pick — the product, the platform, the controls, the test floor. Written so a regulator's technical reviewer or a future engineering hire can read it and walk straight in.



Where we sit on every dimension

136

AUTOMATED CONTROLS
PASSING

6

INDEPENDENT TEST
SUITES

10

REGULATORY FRAMEWORKS
ALIGNED

50pick is a Next.js 16 (App Router, RSC) product on Node.js 22, with PostgreSQL persistence on Railway, full TypeScript strict-mode, scrypt password hashing, HMAC-signed sessions, an HMAC-chained append-only audit log, and a 33-control regulator-aligned automated audit suite that runs against every push. Every state change is logged, every privileged action is server-side gated with defence-in-depth, every error path is bilingual (EN + SW). What follows is how each layer was chosen, what it protects, and how we'd answer a Stage-1 ISO 27001 auditor's first questions.

What this document does NOT contain

Implementation specifics that would help an attacker (rate-limit tuning, scrypt parameters, exact session expiry tolerances, source locations of the audit secret), proprietary betting-pool math (operator-margin algorithm, slippage handling), endpoint enumerations, test-script source. Those live in the private repository under standard need-to-know access controls. This brief is the surface — a reviewer can request specifics under NDA.

SYSTEM ARCHITECTURE

How the pieces fit

EDGE PROXY (NEXT.JS MIDDLEWARE)

Strict CSP · X-Frame DENY · nosniff · Referrer-Policy · Permissions-Policy · COOP · HSTS in prod · attaches request path to the per-request header so server components see the original URL.



APP ROUTER (RSC) — NEXT.JS 16

Server-rendered pages with streaming. Server Actions for every mutation (auth, bet, cashout, KYC, RG). Server Actions are CSRF-bound to the origin and the action ID; payloads are Zod-validated on the server.



IN-MEMORY STORE (RAM)

Read & write hot path. `Map`-backed entities with strongly-typed mirrors of the Prisma schema.



AUDIT RING

Append-only, HMAC-chained. Each entry's hash includes the previous hash → tamper-evident.

↓ debounced 1.5 s

PERSISTENCE — SINGLE SOURCE OF TRUTH

`DATABASE_URL` set → PostgreSQL `StoreSnapshot` row (Railway). Unset → on-disk HMAC-signed JSON envelope. Never both. Boot path rehydrates from whichever is configured.

Why this shape (today)

Single-instance + in-memory + snapshot-to-Postgres lets us ship a soft launch with full ACID-like guarantees for a small bookmaker, without paying the complexity tax of a full multi-instance deploy. Every entity in the in-memory store has a one-to-one Prisma model (19 in the schema) — when traffic warrants horizontal scaling, the swap is method-by-method and tested incrementally. The architecture is not a long-term shape; it is a deliberate near-term shape with a documented migration path.

Where this goes next

Phase 4 of `RAILWAY_DB_README.md` describes the per-entity Prisma swap. After that: read replicas, then a queue-based async write path for the audit log so the chain becomes the slow tier and the request path stays sub-50ms even at peak.

Every choice, defended

NEXT.JS 16 RSC

Server Components by default; Server Actions for every mutation.

Eliminates an entire class of client-trust bugs (the client can't trigger a write without going through a server-validated action). Eliminates a separate API tier — the page *is* the API. Hot-path bundles ship under 80 KB.

TYPESCRIPT STRICT

The schema is the contract. Prisma generates client types from `schema.prisma`; the in-memory store types mirror them exactly. A field added in one place that's missing in another fails at compile.

TAILWIND 3 +
DESIGN TOKENS

Every colour, font, and spacing is a CSS variable defined once in the design kit. There are no hex values inline anywhere a regulator might inspect the CSS. Brand consistency by construction, not by review.

POSTGRESQL +
PRISMA 6

Money handling demands transactional integrity. Prisma's parameterised queries make SQL-injection structurally impossible. Migrations are versioned in git and applied via `prisma migrate deploy` on every Railway deploy.

ZOD VALIDATORS

Every Server Action input is parsed through Zod before reaching the service layer. Bad shapes return a clean `{ ok: false, code }` result, never a 500.

SCRIPT PASSWORD
HASHING

Per-user 16-byte salt. Resistant to GPU brute-force at the parameters used. Common-password blacklist applied before hashing so weak inputs never reach the database.

HMAC-SHA-256
SESSIONS

HttpOnly + SameSite=Lax + Secure-in-prod cookies. Tamper-evident — flip any byte and verification rejects. 7-day absolute cap with a 24-hour idle timeout.

TOKEN-BUCKET RATE
LIMIT

Per (key, action) buckets. Both per-phone and per-IP keys for register/login (defence against credential-stuffing and mass-account abuse). Refills slow enough to stop attackers, fast enough to not annoy real users.

PLAYWRIGHT TEST
HARNESS

136 controls across 6 suites. Real Chromium against a real dev server. Every push runs them locally before commit.

Defence in depth - what stops what

THREAT	LAYER 1 (EDGE)	LAYER 2 (ACTION)	LAYER 3 (SERVICE)
XSS	Strict CSP, X-Frame DENY	React auto-escape, no dangerouslySetInnerHTML on user input	Server-side input sanitisation
CSRF	SameSite=Lax cookie	Server-Action ID binding to origin + form	—
Session theft	HttpOnly + Secure cookie	HMAC verify on every request	Idle timeout + absolute expiry
Brute force	—	Per-phone + per-IP rate-limit	Account lockout after N failures
Privilege escalation	Edge gate on /admin/*	Layout-level role check	Per-action role assertion (defence in depth)
SQL injection	—	Zod input validation	Prisma parameterised queries
Replay	—	Server Action ID is per-build; expired actions reject	Idempotency keys on financial writes
Audit tamper	—	—	HMAC chain — any modified entry breaks the chain
PII leak	Phone masked in UI	No PII in error messages	No PII in /api/health or audit-log payload fields
Mass-account abuse	—	Per-IP register cap	Common-password blacklist · 18+ DOB enforcement

Audit-log integrity (the regulator question)

Every state change emits one entry: `category` , `action` , `actorId` , `targetType` , `targetId` , `payload` , `ip` , `userAgent` , `createdAt` , plus `prevHash` and `entryHash` . `entryHash` is HMAC-SHA-256 over the canonical JSON of the row including `prevHash` . The chain is verifiable in O(n): walk

forward, recompute each hash, compare against the stored one. Any modified or removed entry breaks at the affected index — the verifier reports the first break, which is what a forensic reviewer needs.

Frameworks & how we satisfy them

FRAMEWORK	DOMAIN	HOW 50PICK ALIGNS
LCCP (UK Gambling Commission, Licence Conditions & Codes of Practice)	Player protection, AML, integrity	SR Code 2.4 (RG controls), 5.1.5 (visible 18+, helpline, RG link on every page), 2.6 (account security), 2.10 (audit-trail), 2.13 (financial integrity), 2.14 (anti-fraud).
GBT (Gaming Board of Tanzania)	National licensing	Pre-application packet drafted. Pool model classified as betting pending GBT confirmation. Helpline + license-ref on every page footer.
ISO 27001	Information security mgmt	A.9 access control (RBAC + role gates), A.12.3 backup (HMAC-signed snapshot, daily Postgres backup), A.12.4 logging (audit chain), A.18.1 compliance (this document).
OWASP Top 10 (2021)	Web application security	A01 Broken access control — defence-in-depth role checks. A02 Cryptographic failures — scrypt + HMAC. A03 Injection — Prisma parameterised + Zod. A04 Insecure design — threat model documented. A05 Security misconfig — strict CSP + nosniff + HSTS. A07 ID/auth — rate-limit + lockout + idle timeout. A09 Logging — audit chain.
PDPA (Tanzania, 2022)	Personal data protection	Phone masked in UI. PII never in /api/health, error messages, or audit payloads. Right-to-export and right-to-erasure stubs in /admin/privacy.
FATF Recommendations	AML	Transactions over threshold queue at /admin/aml. Compliance-officer review with documented reason. Audit-trail retention for the FATF retention window.
TCRA	Telecommunications	SMS sender-ID licensing path documented. Outbound SMS gated by provider abstraction; provider switch is config-only.

FRAMEWORK	DOMAIN	HOW 50PICK ALIGNS
WCAG 2.1 AA	Accessibility	Colour-contrast OKLCH-driven. Every interactive element has aria-label or visible label. Keyboard navigation throughout. Reduced-motion respected.
OWASP ASVS L2	Verification standard	Targeting Level 2 (most controls already met). Level 3 reachable post-Phase-4 Prisma swap.
RFC 6238	TOTP MFA	Admin TOTP is implemented; setup at /admin/2fa/setup; verification gate at /admin/totp-verify.

QUALITY ASSURANCE

Test coverage — 136 / 136 controls

Every control runs in headless Chromium against a real Next.js dev server. Every push runs the full suite locally before commit. The suites are independent — one suite's failure tells the engineer exactly which layer broke.

SUITE	SCOPE	RESULT
REGULATOR-AUDIT	33 controls across 10 sections — auth security, authorization defence-in-depth, player protection, PII, financial integrity, audit log, security headers, input validation, anti-fraud, regulator-required UI	34 / 34
SPRINT35-FULL-E2E	Real OTP-style registration → 3 bets at varying stakes → break-it block (triple-click guard, cancel mid-flight, quote-expire) → kit-spec popups	15 / 15
BREAK-IT-ADMIN	21 admin routes gated for anon + player. TOTP cookie spoofing rejected. Forged Server-Action POSTs from a player do not write market.* audits. Speculative /api/admin/* paths return 4xx. Anon CSV-export paths return no CSV.	10 / 10
BREAK-IT-PLAYER	10 sections, 31 manipulator scenarios — auth bypass, cookie tamper, stake validation (negative / NaN / Infinity / MAX_SAFE_INTEGER / fractional / over-balance), market validation, race / replay, KYC bypass, XSS, privilege escalation, large payloads, time tampering, bot patterns.	31 / 31
DEMO-ACCOUNT-FLOW	End-to-end manager-demo player walk: register success / failures, login success / failures (including lockout after 5 wrong tries), forgot-password reachable, phone digits-only formatter, password strength meter, logout confirm dialog.	19 / 19
DEMO-ADMIN-FUNCTIONAL	Logs in as a real admin and walks every one of 19 admin pages, asserts each renders 200, carries the AdminShell confidentiality band, and has no body-level error. Plus 4 functional spot-checks: audit-log entries visible, player-list metrics visible, system-page rate-limit visible, role-tone band shows.	27 / 27
Total		136 / 136

What's NOT tested today

- Real SMS delivery (provider not yet contracted — adapter abstraction is in place).
- Real mobile-money deposit / withdrawal end-to-end (provider not yet contracted — same abstraction shape).
- Production Postgres failover (Hobby tier is single primary; reachable as soon as we move to Pro + read-replica).

- Load test at 6M clicks/month projected target — the existing test harness is correctness, not load.

19 entities, drawn at the right altitude

The entity boundaries match the regulator's mental model: identity (User, Session, OTP, Device), KYC, money (Wallet, Transaction), betting (Market, MatchEvent, Window, Pool, BetBundle, Bet), integrity (AntiFraudFlag, MatchIntegrityCheck), compliance (ResponsibleGambling, Notification, AuditLog), and partners (AffiliateAgent, ProviderHealth). A regulator asks for a bet's audit trail; we walk the chain in three steps.

Why these specific lines

- **User** is identity + role. KYC, RG, Wallet are separate entities with FKs back. Avoids the temptation to bloat User and lose composability.
- **Wallet** is one-to-one with User. Holds `balance` + `pending` + `hold` separately so an in-flight withdrawal cannot be double-spent.
- **Transaction** is the immutable ledger entry. Status transitions only forward (PENDING → CONFIRMED / FAILED). Reversal is a NEW transaction.
- **Bet** belongs to a Market via a BetBundle (multi-market accumulator). Stakes are integer TZS — no floating point.
- **Market** has resolution-source URL + two-officer sign-off fields baked in. A market without these can't be created.
- **AuditLog** is append-only. `prevHash` + `entryHash` form a Merkle-style chain. Verifiable in $O(n)$.

Why these are NOT shown here

- Field-level types and enums.
- Index choices.
- The pool-share calculation algorithm.
- The exact AML threshold.
- Audit-chain HMAC key location.

These live in the private repository. Engineering hires get day-one access; partners get specifics under NDA.

How we ship · how we scale

Today (soft-launch shape)

Single Railway service running Next.js. Single Railway-managed PostgreSQL. `git push` triggers a redeploy in 2–3 minutes; `prisma migrate deploy` runs automatically on every build, so a schema change is one commit + one push. Daily Postgres backups on Railway with 7-day retention; weekly external archive to S3 Glacier Deep Archive for the 5-year regulator retention window.

Next (production hosting)

- **Per-entity Prisma swap** (`store.ts` → 10 Prisma services). 60–90 min of focused work per entity slice; runbook in `RAILWAY_DB_README.md` Phase 4.
- **Read replica** for analytics + admin dashboards.
- **Connection pooler** (Railway's pgbouncer endpoint already wired; one env-var swap).
- **Multi-instance + sticky sessions** behind a load balancer. The session cookie is signed not stored, so any instance can verify any session — no shared cache needed.
- **Async audit-log writer** (queue + worker). The chain becomes the slow tier; the request path stays sub-50 ms even at peak.
- **External SMS** (Selcom or Beem) and mobile-money aggregator wired through the existing provider abstractions — flag flips, no code change.

Observability runbook

`/api/health` exposes uptime, user count, audit-ring size, market counts, and SMS provider success rate — Railway's built-in health check polls it. Application logs go to Railway's log retention (7 days on Hobby, 30 on Pro). Each request log line carries a request-id; an audit entry that points to a request-id can be joined against the application log for full forensic context.

What a partner sees on day-one engagement

A handover packet that's already drafted: this brief + the operator brief + the design kit + the Prisma schema + the regulator-audit test suite. A senior engineer can read those and ship a feature in a week without asking us a single onboarding question.